

extitArticle/Review

extbfABC – Automatic Beep Censor

extbfNutchapol Rodpholchoo, Nuchanon Tongkumbanchong, and Sirinalin Saekhow

Vincent Mary School of Engineering, Science and Technology, Assumption University

E-mail: nutchapol.ro@student.au.edu, nuchanon.to@student.au.edu, sirinalin.sa@student.au.edu

Abstract. This project investigates profanity detection in Thai language audio, addressing the challenges of limited data availability we faced. While initial model development was constrained by data scarcity, the focus shifted to a comprehensive evaluation of detection methods and parameter optimization. By systematically analyzing various evaluation metrics and window/stride configurations, the study provides practical guidance for future research and model improvement, enabling others to efficiently optimize profanity detection systems even with limited resources.

Keywords: Thai profanity detection, Wav2Vec2, window stride optimization, IoU, VAD, evaluation methodology

extbfENGINEERING JOURNAL Volume # Issue #

Received Date Month Year

Accepted Date Month Year

Published Date Month Year

Online at <http://www.engj.org/>

DOI:10.4186/ej.20xx.xx.x.xx

1. Introduction

Content moderation remains a critical challenge for digital media platforms which contain hundreds of explicit contents, especially in languages with limited computational resources such as Thai. While automated profanity detection systems are well-researched for languages like English for text detection [?], and audio detection [?]. Thai presents unique computational challenges due to its tonal nature, complex phonetic structure [?], and the scarcity of large-scale profanity-labeled datasets. Moreover, traditional manual censorship is still widely used, but it is time-consuming, inconsistent, and impractical for large-scale content moderation.

Recent advances in speech recognition and deep learning have enabled more sophisticated content moderation tools. However, most existing models are trained on non-Thai datasets, resulting in high error rates when applied to Thai profanity detection. Our work addressed these challenges by fine-tuning a pre-trained Wav2Vec2 model [?] specifically for Thai profanity detection through supervised learning with sequence classification. The base model used was `airesearch/wav2vec2-large-xlsr-53-th`, which had been pre-trained on Thai speech data using self-supervised learning techniques.

While this initial system demonstrated promising results, its performance was constrained by limited training data availability, with restricted time and computational resources. The fundamental issue in low-resource language processing is that deep learning models typically require extensive labeled datasets to achieve optimal performance, which may be impractical to obtain for specialized tasks like profanity detection [?].

1.1. Problem Statement

The development of robust Thai profanity detection models faces a fundamental constraint: the lack of extensive labeled datasets that would typically be required for optimal performance. While ASR-based solutions show promise, they struggle with Thai's phonetic diversity, slang variations, and tonal ambiguities, leading to inconsistent profanity detection accuracy. More critically, the evaluation methodologies used to assess these systems often lack systematic analysis of temporal parameters, specifically window size and stride configurations, that significantly impact detection performance in practical deployment scenarios.

Traditional evaluation approaches in speech processing typically use fixed parameters without exploring the relationship between temporal segmentation settings and detection accuracy. In profanity detection, the choice of analysis window size and stride directly affects both detection sensitivity and temporal precision, yet existing

evaluation methods rarely provide appropriate stride for parameter optimization [?]. Especially for Thai language, which has differences in phoneme and tone.

Recognizing the data limitation as an inherent challenge rather than a solvable problem within the scope of our research, this project shifts focus from purely model development to comprehensive evaluation methodology and parameter optimization. The central question becomes: Given limited training data, how can we systematically identify the most effective evaluation approaches and optimal system parameters (window size and stride configurations) to guide future research and practical implementation?

1.2. Scope of the Project

This project centers on the comprehensive evaluation and optimization of Thai profanity detection systems in audio content. Building upon the foundational fine-tuned Wav2Vec2 model developed in Senior Project 1, we design and implement multiple evaluation methodologies to assess detection performance and meaningful evaluation results.

The research systematically explores the impact of critical temporal parameters. Specifically, window size (ranging from 0.3s to 2.0s) and stride configurations (in percentage of window length and changing stride every 0.05 seconds) on detection accuracy across different parameter combinations.

1.3. Significance

This research addresses a critical practical need in Thai profanity detection by systematically identifying the optimal window and stride parameters for different evaluation methods. The primary significance lies in providing clear, evidence-based recommendations for future researchers and practitioners working on similar Thai language processing tasks, eliminating the need for time-consuming trial-and-error parameter selection.

The project's main contributions include:

- **Optimal Parameter Identification:** Through systematic analysis of window and stride combinations across evaluation methods, the study identifies the best-performing configurations for each evaluation approach, providing concrete guidance for future model implementations.
- **Evaluation System Framework:** The comprehensive evaluation system developed can be directly applied by future researchers to assess their own Thai profanity detection models, ensuring consistent and reliable performance measurement across different implementations.

The practical impact of this work enables future developers to immediately implement optimal configurations (such as 0.3s windows for word-level tasks) rather than spending significant time on parameter exploration, accelerating the development of Thai language content moderation systems in large datasets.

2. Related Work

Profanity detection in speech has been a significant research area in natural language processing (NLP) and speech processing. Existing methods generally fall into three main categories: Automatic Speech Recognition (ASR)-based text processing [?, ?], direct speech-based profanity detection using deep learning [?, ?], and text-based hate speech research [?] or keyword-based approaches [?]. This section provides an overview of these approaches and their relevance to profanity detection in Thai speech, with particular emphasis on evaluation methodologies and parameter optimization techniques that inform our research approach.

2.1. ASR-Based Profanity Detection

Traditional approaches to profanity detection in speech typically involve a two-stage process: first converting speech to text using Automatic Speech Recognition (ASR) systems, then applying text-based profanity detection algorithms [?]. While this approach benefits from mature text processing techniques and extensive text-based profanity datasets, it is unable to locate timestamps in audio precisely.

2.2. Direct Speech-Based Profanity Detection

Recent advancements in deep learning have enabled direct processing of raw audio for speech classification, bypassing the need for ASR. Convolutional Neural Networks (CNNs) and Recurrent Neural Networks (RNNs) have been applied to spectrogram representations of audio signals for emotion and profanity classification [?]. More recently, self-supervised learning models like wav2vec 2.0 [?].

Wav2Vec2 architecture offers significant advantages over ASR-based methods for profanity detection through its direct waveform analysis capabilities. Unlike ASR systems that convert speech to discrete text tokens, Wav2Vec2 processes raw audio waveforms directly, enabling precise temporal localization of acoustic features [?]. This direct analysis approach allows for fine-grained windowing strategies that can capture the exact temporal boundaries of profane words, which is crucial for accurate detection and subsequent censoring applications. The model's ability to analyze speech at various temporal resolutions through configurable window sizes makes

it particularly suitable for profanity detection tasks where precise temporal localization is essential for practical deployment.

2.3. Evaluation Methodologies in Speech Processing

2.3.1. Window Evaluation

A common methodology for analyzing continuous audio streams is window-based evaluation. This approach involves segmenting the audio into fixed-size windows, which are then individually classified.

A pertinent example is the work of Wazir et al. [?], who investigated profane word detection for film censorship. They systematically analyzed the effect of window length by testing segments of 0.3, 0.4, and 0.5 seconds. Their findings indicated that model performance, measured by metrics such as F1-score and accuracy, improved with longer segments, leading them to conclude that 0.5 seconds was the optimal window length for their system.

Furthermore, Wazir et al. addressed the inherent problem of fixed windowing where words can be split across two segments. They introduced an "overlap time" (a stride of less than 100%) and demonstrated that using overlapping windows significantly improved detection accuracy by reducing false negatives.

While their study established a clear benefit for a specific window size and the use of an overlap, it did not perform a broad analysis of different stride values. This leaves an open question regarding the optimal relationship between window size and stride percentage, which is a central focus of our research.

2.3.2. Frame Evaluation

A more granular approach to audio event detection is frame-level evaluation, as demonstrated in the work on explainable hate speech detection by An et al. (2023). In this methodology, the audio signal is segmented into extremely small, fixed-size "frames," typically around 10 milliseconds.

Unlike window-based methods that classify a larger segment of audio, a frame-level model directly analyzes the raw audio waveform and makes an independent prediction for each individual frame. The primary goal is to generate a detailed, time-stamped sequence of classifications, often referred to as a "rationale," which pinpoints the exact temporal locations of the event of interest (e.g., hate speech).

The performance of this method is not just measured by classification accuracy but, more importantly, by localization metrics. An et al. (2023) utilize the Intersection over Union (IoU) metric, which evaluates how well the predicted sequence of positive frames overlaps with

the ground-truth sequence. This makes frame-level evaluation particularly powerful for tasks requiring high temporal precision. However, this approach relies on having datasets with highly detailed, frame-by-frame annotations, which are often resource-intensive to create. This contrasts with our work, which investigates the optimal parameters for a window-based approach that is more practical for datasets without such fine-grained labels.

Both frame-level and window-based evaluation methodologies offer distinct advantages for speech processing tasks. Frame-level analysis provides unparalleled temporal precision, making it ideal for tasks requiring exact localization and explainability. However, its primary prerequisite is the availability of datasets with exhaustive, frame-by-frame annotations. Creating such detailed labels is extremely resource-intensive and often impractical, especially in the context of low-resource languages like Thai.

In contrast, window-based evaluation presents a robust and highly effective alternative that functions well with more commonly available datasets, which typically have word-level or clip-level labels. This approach aligns perfectly with the primary objective of this research: to determine the optimal temporal parameters for a practical profanity detection system. Therefore, for this project, we have adopted window-based evaluation methodology. This strategic choice allows us to conduct a large-scale, systematic analysis of how different window and stride configurations impact detection performance, without being constrained by the need for fine-grained, frame-level annotated data. Our focus is on optimizing a practical and deployable system, making the window-based approach the most suitable and logical choice.

2.4. Challenges in Thai Speech Profanity Detection

2.4.1. Linguistic Characteristics

Thai presents unique computational challenges due to its tonal nature, complex phonetic structure, and lack of explicit word boundaries in written text. These characteristics directly impact speech processing systems, requiring specialized approaches for effective analysis. The tonal nature of Thai means that subtle variations in pitch can change word meanings, adding complexity to automatic processing systems and potentially affecting optimal parameter choices [?].

2.4.2. Phonetic Complexity and Parameter Impact

The complex phonetic structure of Thai, including its tonal characteristics and phoneme variations, suggests that optimal temporal parameters may differ significantly from those established for non-tonal languages [?]. This linguistic specificity necessitates dedicated parameter op-

timization studies for Thai language applications rather than adopting configurations optimized for other languages.

2.4.3. Limited Dataset Availability

Thai language processing suffers from limited availability of large-scale labeled datasets, particularly for specialized tasks like profanity detection. This constraint necessitates efficient approaches to model development and evaluation that can provide maximum insight with minimal data requirements, making parameter optimization studies particularly valuable for maximizing the utility of available data.

3. Proposed Methodology

3.1. Data Collection

We collect data by recording people's voices and downloading video and audio content around the internet. Then we label profanity with the Audacity application, an audio editor. Each label includes start time, end time, and the profane word. After that, we use a CSV file to list all the labels and file paths. An example schema is shown in Table 1.

file_path	start_time	end_time
./dataset/audio/Jan-says-about-Art.wav	0	14.18
./dataset/audio/Jan-says-about-Art.wav	14.18	14.91
./dataset/audio/Jan-says-about-Art.wav	14.91	15.37

Table 1. Example of saved label data.

3.2. Data Processing

We process data by normalizing audio and resampling to 16 kHz and converting from stereo to mono. Moreover, we reduce noise using spectral gating techniques. We window or segment audio by using start time and end time of profanity in training. However, in real usage and test, we create windows and move every 0.5 second, but with overlapping technique [?] it will improve accuracy of profanity detection. For instance, the window frame is 0–0.5 second at the starting point. So, instead of moving next to 0.5–1.0 it will go to 0.25–0.75 instead to prevent the audio wave gap that might split or distort the actual profanity waveform.

3.3. Model Architecture

3.3.1. Model Selection

After researching several approaches, the Wav2Vec 2.0 model [?] was selected for its strong performance on speech recognition tasks, ability to work with waveforms,

efficiency, and effective feature extraction from raw audio.

3.3.2. Architecture Details

The model architecture consists of several key components:

1. Input Layer: Accepts raw audio waveforms (converted to 16,000 samples per second at 16 kHz), 16-bit; no manual feature extraction required.
2. Feature Encoder: Multi-layer convolutional neural network that extracts audio features from raw waveform (example output shape: 49×512 time steps by features).
3. Feature Projection: Linear projection layer that transforms features to match transformer dimensions (e.g., 49×1024).
4. Transformer Encoder: 12 transformer layers with multi-head attention to capture contextual relationships in audio; output shape 49×1024 .
5. Mean Pooling Layer: Aggregates sequence information across time to produce a fixed-length representation (1024).
6. Dropout Layer: Rate of 0.1 for regularization.
7. Classification Layer: Dense layer with 4 output neurons and softmax activation (4 profanity types + none).

3.4. Training Approach

3.4.1. Training Strategy

We began with a pre-trained Wav2Vec 2.0 model `airesearch/wav2vec2-large-xlsr-53-th`, which had already learned rich representations from general speech in Thai. We used a batch size of 16, a learning rate of 5×10^{-5} with linear decay, and class weighting to address class imbalance. We trained for up to 100 epochs with early stopping using AdamW with weight decay. We also padded the start and end of each profanity window by 0.2 seconds to capture transitions.

We used the weighted cross-entropy loss:

$$\mathcal{L}_{\text{weighted}} = - \sum_{c=1}^C w_c y_c \log p_c, \quad (1)$$

$$w_c = \frac{N}{C N_c}, \quad (2)$$

where N is the total number of samples, C is the number of classes, and N_c is the number of samples in class c .

Data augmentation included: time stretching ($\pm 10\%$), pitch shifting (± 2 semitones), background noise at various SNRs, and random time masking. We used an 80/20 split for training and testing with stratified sampling and monitored performance on the test split for early stopping.

3.4.2. Hyperparameter Optimization

Our work centers on the critical pre-processing stage that defines how the model sees the audio data: the windowing and stride configuration. We executed an exhaustive parameter sweep, evaluating many window/stride combinations.

Window Size: We tested window lengths from 0.3 s to 2.0 s to understand the trade-offs between temporal precision and contextual understanding.

Stride Configuration: For each window size, we evaluated both percentage-based overlaps (from 10% to 90%) and fixed-time overlaps (e.g., moving every 0.05 s).

Each parameter combination was assessed across four distinct evaluation methodologies: merged word evaluation, word IoU evaluation, IoU evaluation, and window evaluation. This provides a nuanced understanding of how window and stride parameters affect different aspects of detection performance, from simple classification to precise temporal localization.

3.5. Evaluation Methodology

Our evaluation framework assesses performance under two striding strategies: (1) fixed-time stride of 0.05 s and (2) percentage-based stride relative to window length. We employ four methods:

3.5.1. Window Evaluation

Ground truth is defined at the window level. A window is labeled profanity if its overlap with any labeled profane word is $\geq 50\%$. Model window predictions are compared directly against these labels to compute Accuracy, F1, etc.

3.5.2. Merged Word Evaluation

Consecutive windows predicted as the same profanity class are merged into a single continuous segment. For example, a ground-truth profanity from 2.00–3.25 s might be reconstructed from overlapping 0.5 s windows at 0.25 s stride as a single predicted event from 2.00–3.25 s. This tests the model's ability to detect complete profanity events.

3.5.3. Merged Word IoU Evaluation

This enhances merging by using IoU between the merged prediction and ground-truth word as a criterion for TP/FP. A merged prediction is a true positive only if its IoU with a ground-truth word exceeds a threshold (e.g., 0.5). Inconsistent predictions separated by gaps form distinct instances, each evaluated independently.

3.5.4. Merged Word IoU Area Combination Evaluation

To be completed: integrate detailed criteria for area combination and, if applicable, the role of VAD in refining boundaries.

4. Design of the System

4.1. System Design

This section will detail the overall system design including data ingestion, preprocessing, inference, post-processing, and evaluation subsystems.

extitTODO: Insert finalized system architecture diagram and description.

4.2. Website Architecture and Implementation

Outline the web application architecture used to demonstrate profanity detection and censoring.

extitTODO: Describe backend, frontend, storage, and integration points.

4.3. Technical Stack and Components

extitTODO: List frameworks, libraries, deployment environment, and configurations.

4.4. Processing Workflow

extitTODO: Provide step-by-step processing flow from upload to censored output with timing considerations.

4.5. UI/UX

extitTODO: Add UI/UX screenshots and describe user interactions.

5. Results

5.1. Model Performance

5.1.1. Evaluation Methodology Selection

extitTODO: Summarize why certain evaluation methods are preferred based on experiments.

5.1.2. Finding Appropriate IoU Threshold

extitTODO: Present analysis across thresholds for both binary and multiclass F1.

5.1.3. Window Size Optimization Performance

extitTODO: Include performance across window sizes (0.3–2.0 s).

5.1.4. Finding Appropriate Stride

extitTODO: Compare percentage-based vs fixed 0.05 s stride.

5.1.5. IoU Percentage

extitTODO: Provide mean IoU heatmaps and interpretation.

5.2. Real-World Usage Recommendations

extbfBinary Profanity Detection for Censoring Applications

extitTODO: Provide recommended configs with rationales for production censoring.

6. Future Work and Limitation

6.1. Future Work

extbfContext-Aware Detection using Hierarchical Models

extbfAdvanced Boundary Refinement with Neural VAD

extbfReal-Time Streaming Architecture

extitTODO: Expand each item with approach and expected impact.

6.2. Limitation

extitTODO: Discuss dataset size, labeling granularity, and generalization limits.

7. Conclusion

extitTODO: Concise summary of contributions, findings, and actionable takeaways.

Acknowledgments

We thank Dr. Kwankamol Nongpong for advising this project.